

数字集成电路与系统设计

课程导论

计算机

系统概述

教师

计算机与软件学院高性能研究所

蔡晔

caiye@szu.edu.cn

L6-509办公室

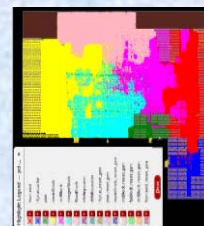
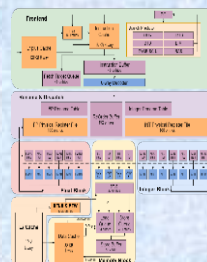
主要研究方向

■ 研究方向：开源处理器芯片、芯片人才培养、开源智能EDA

开源 处理器 芯片



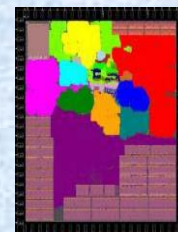
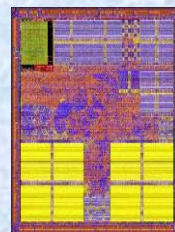
- 香山高性能开源RISC-V处理器核
- 雁栖湖、南湖、昆明湖三代
- 国际性能最好的开源处理器芯片



芯片 人才培 养



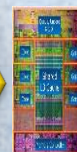
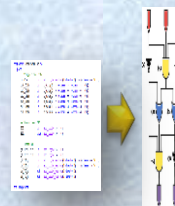
- www.ysyx.org
- 从零开始创造属于你自己的处理器
- 从设计到流片全流程



开源智 能 EDA



- 面向国产28nm工艺
- 构建全链条芯片设计的开源EDA平台



开源EDA平台

教材

。 自编教学与实验教材，主要分为三大块：

1、基于Chisel的硬件描述语言（HDL: Hardware Description Language）

实验1-9 必做（前8周完成）

现场验收评分+提交实验报告（畅课平台LMS）

2、课程设计(GRADE A, 9-18周完成)

课程设计1：芯片前端设计流程(LC-3处理器设计)

从HDL到Netlist（网表:用于描述电路元件相互之间连接关系的文件），最终通过FPGA实现（GRADE A）

程设计2：芯片后端设计流程(LC-3处理器后端设计)

从Netlist到GDSII（Graphic Data System II，芯片版图布局文件：包含了完整的芯片版图布局信息，例如晶体管、连线、金属层等设计。可用于制作光刻掩膜版）

完成指标：

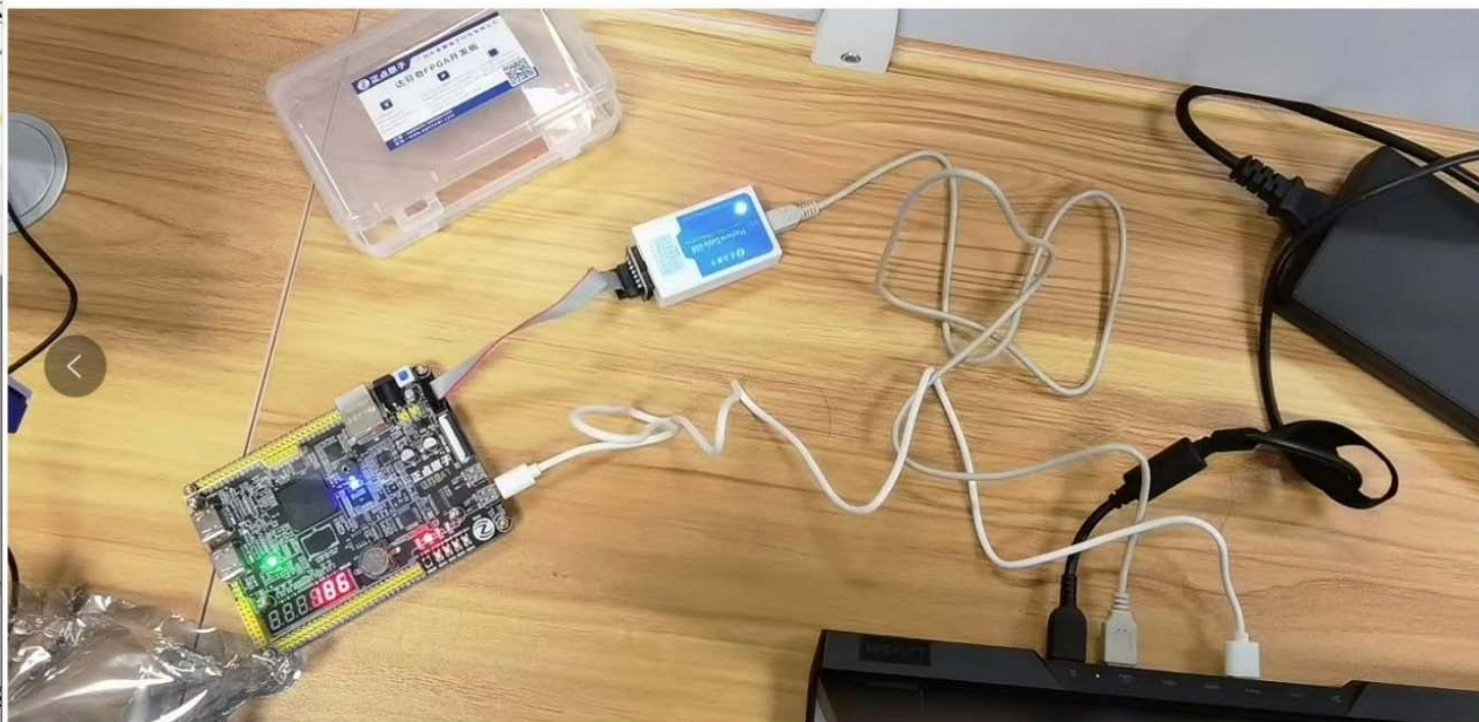
每个实验：现场验收评分+实验报告（LMS）

课程总体报告1份：课程设计报告（LMS）+纸质版（课程考核）

3、附加设计题（GRADE A+:10%）

现场验收评分+提交实验报告（LMS）

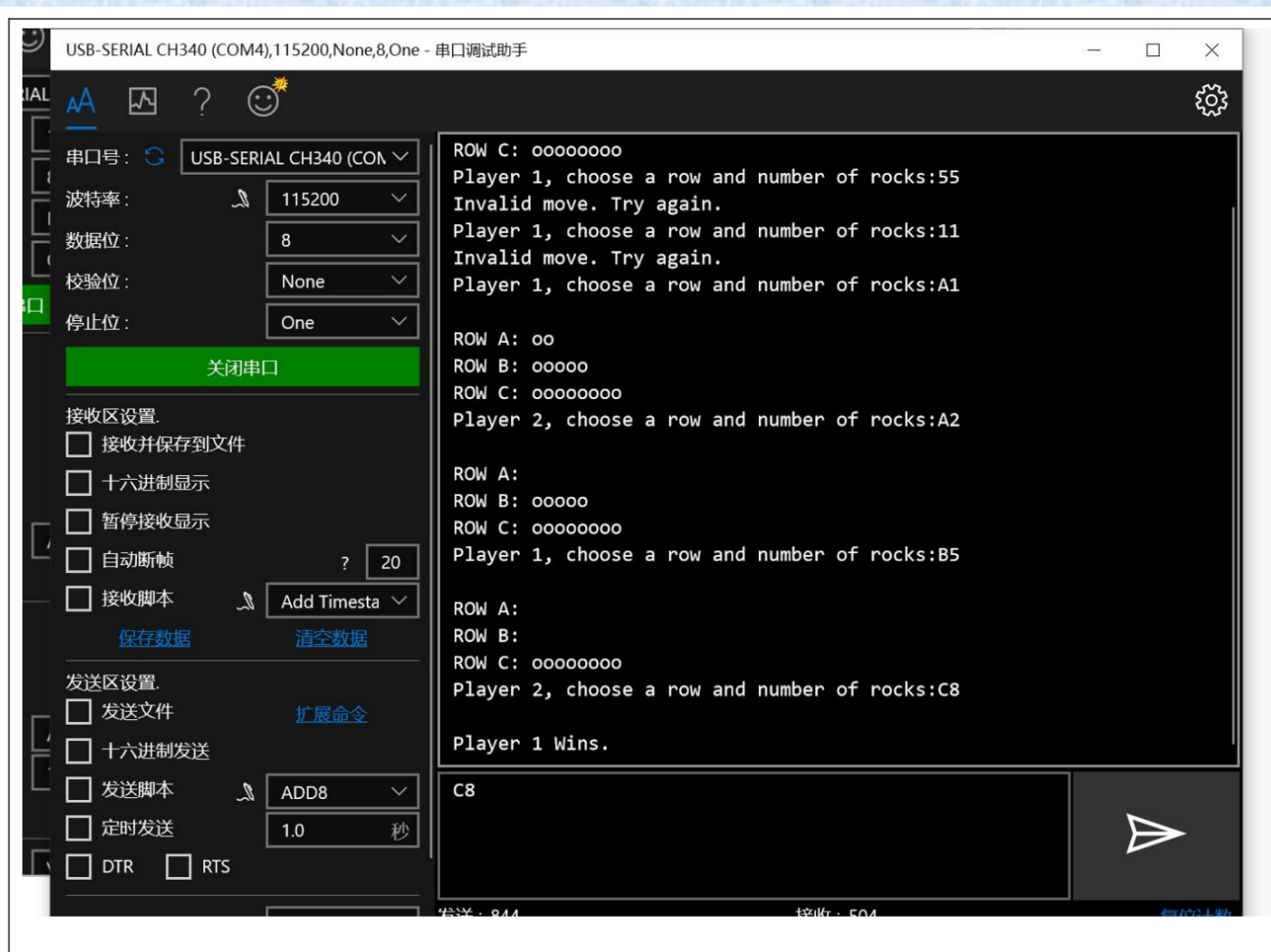
课程目标：自己实现LC-3处理器



点击 Hardware 窗口中的 Auto Connect 按钮。看到如图所示的画面，代表已经连接成功了。

```
1 create_clock -period 20.000 -name clk [get_ports clock]
2 set_property -dict {PACKAGE_PIN R4 IOSTANDARD LVCMOS33} [get_ports clock]
3 set_property -dict {PACKAGE_PIN T1 IOSTANDARD LVCMOS33} [get_ports reset_n]
4 set_property -dict {PACKAGE_PIN U5 IOSTANDARD LVCMOS33} [get_ports io_uart_rxd]
5 set_property -dict {PACKAGE_PIN T6 IOSTANDARD LVCMOS33} [get_ports io_uart_txd]
6
7 set_property CFGBVS VCCO [current_design]
8 set_property CONFIG_VOLTAGE 3.3 [current_design]
9 set_property BITSTREAM.GENERAL.COMPRESS true [current_design]
10 set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
11 set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
12 set_property BITSTREAM.CONFIG.SPI_FALL_EDGE Yes [current_design]
```

在自己实现的LC-3上 运行简单游戏程序



1、Chisel Bootcamp

(<https://mybinder.org/v2/gh/freechipsproject/chisel-bootcamp/master>)是一个很不错的chisel教程, 还支持在线运行chisel代码, 你可以一边编写chisel代码一边学习. 其中

- * 第1章是scala入门
- * 第2章是chisel基础
- * 第3章是scala高级特性和chisel的混合使用
- * 第4章是FIRRTL后端相关内容 你需要完成前两章的学习, 同时我们强烈建议你学习第3章. 第4章和本课程没有直接关系, 可以作为课外阅读材料.

2. Chisel Users Guide(<https://www.chisel-lang.org/docs>)比较系统地整理了chisel的特性, 也是不错的入门教程.

3. Chisel小抄(https://github.com/freechipsproject/chisel-cheatsheet/releases/latest/download/chisel_cheatsheet.pdf)简明地列出了chisel语言的大部分用法.

4. Digital Design with Chisel(<http://www.imm.dtu.dk/~masca/chisel-book.html>)是一本将数字逻辑设计和Chisel结合的参考书

5. Chisel API(<https://www.chisel-lang.org/api/latest/>)详细地列出了chisel库的所有API供参考.

教学模式：实践驱动教学

- ▶ 每周四节课连上，**1+3**模式实践驱动教学
- ▶ **1+3**：教师在第**1**节课进行引入（**5-10**分钟），剩余时间进行实验学习并完成实验。
- ▶ 通过实验进行教学，由实验推动教学

课程考核方式

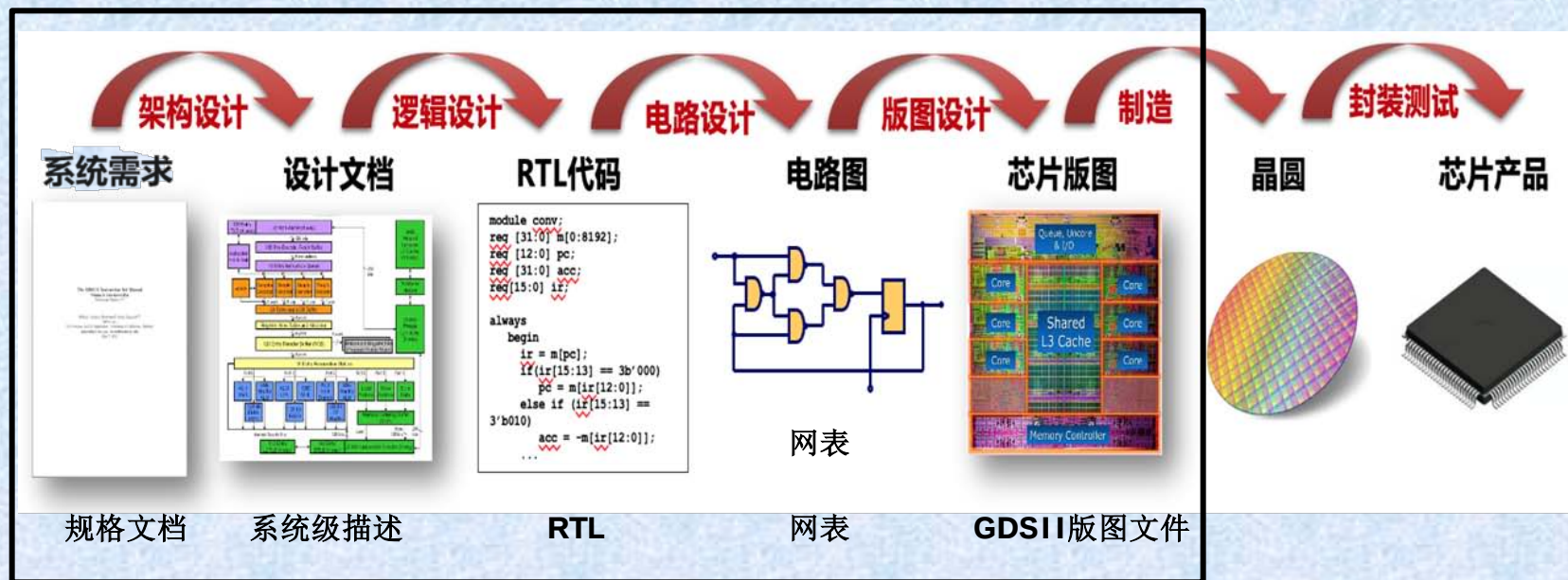
- 1、实验：每个实验要求独立完成，每次课程需进行实验签到。实验课有实验助教。实验完成后需向助教申请验收并评分，登记完成与否。
- 2、每次实验完成后实验报告（**40%**），**BB**上提交
- 3、考核：课程设计报告，**60%**
- 4、学术诚信：不接收任何理由的迟交实验报告。发现抄袭，课程成绩直接为**F**。
- 5、**A**标准，正确完成前端设计和后端设计流程，并在**FPGA**上实现**CPU**，**A+**需要完成附加设计题。

数字芯片设计流程

前端设计

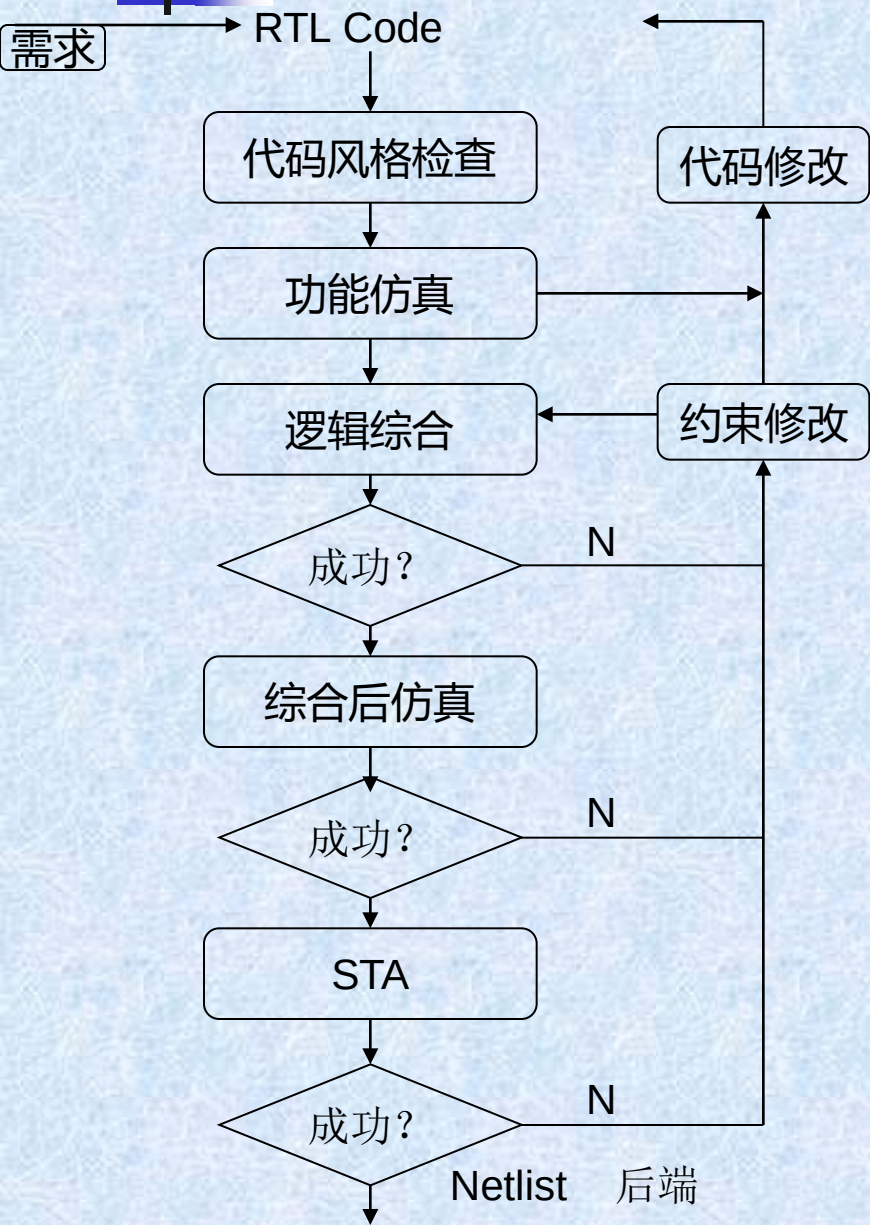
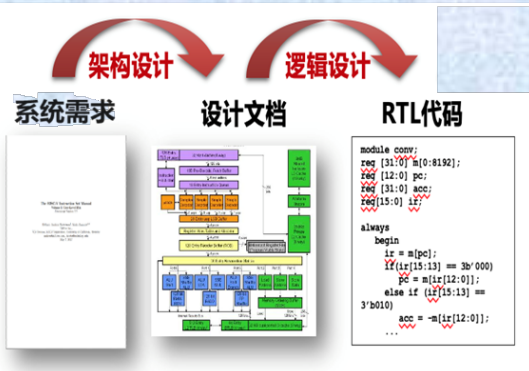
后端设计

芯片制造



芯片设计流程

芯片



RTL (Register Transfer Level) 设计

利用硬件描述语言，如 chisel/verilog，对电路以寄存器之间的传输为基础进行描述

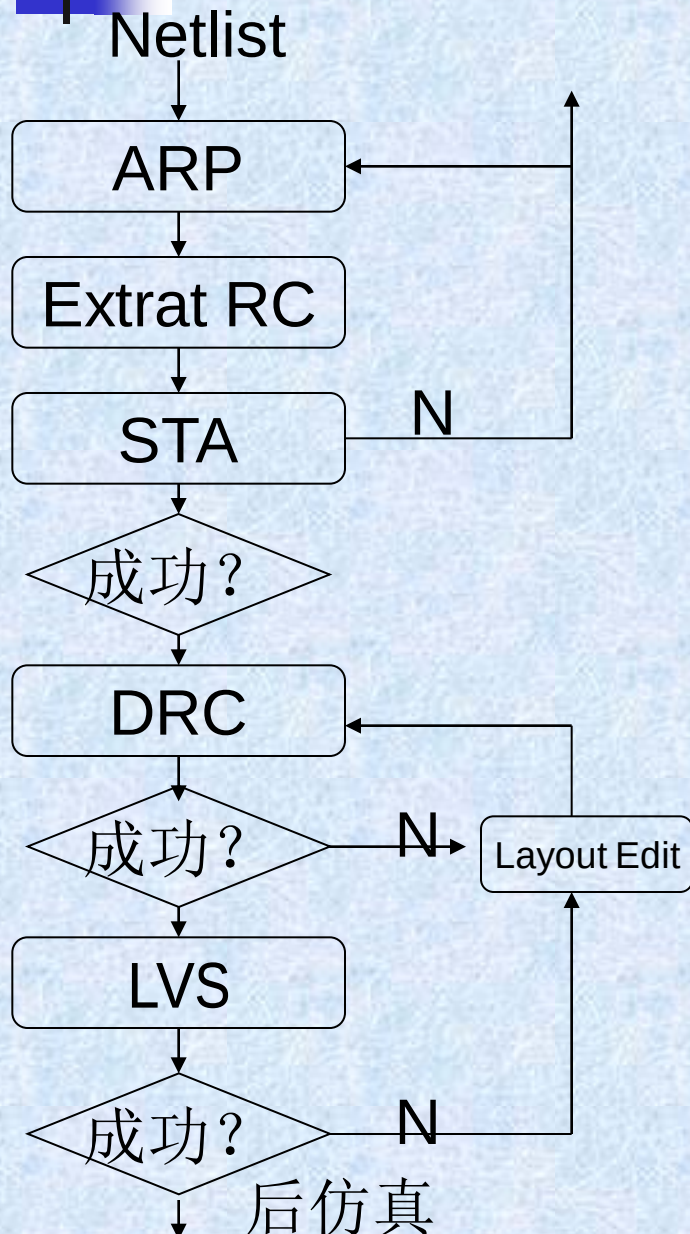
综合：

将RTL级设计中所得的程序代码翻译成实际电路的各种元器件以及他们之间的连接关系，可以用一张表来表示，称为门级网表 (Netlist)。

STA (Static Timing Analysis, 静态时序分析)：套用特定的时序模型 (Timing Model)，针对特定电路分析其是否违反设计者给定的时序限制 (Timing Constraint)

整个前端设计流程都是一个迭代的流程，在任何一步不能满足要求，都需要重复之前步骤，甚至重新设计RTL代码。

后端设计 (Netlist to Layout)

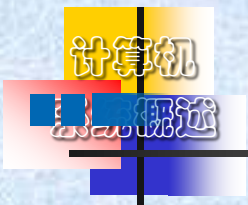


APR:Auto Place and Route,自动布局布线

Extract RC:提取延时信息

DRC: Design Rule Check, 设计规则检查。

LVS: Layout Versus Schematic, 版图电路图一致性检查。



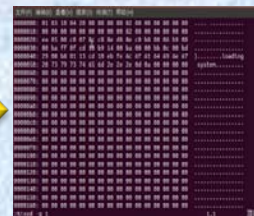
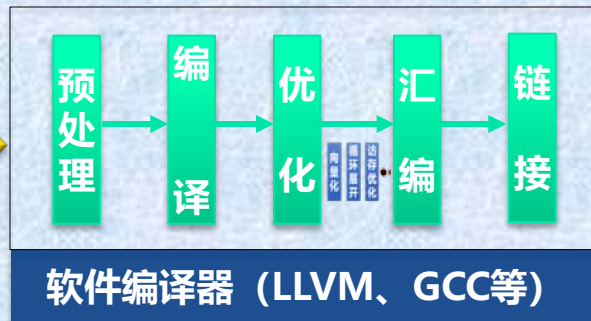
芯片设计工具— EDA工具

■ 基于EDA工具的设计开发流程：自动化的芯片设计流程及其辅助工具，提升芯片设计效率和质量

软件设计流程 gcc -O3

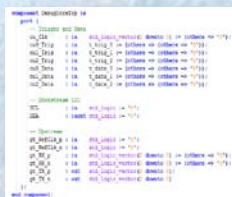


C、C++/Java

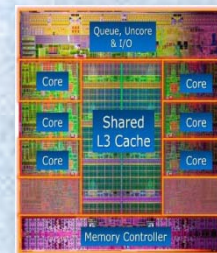


完备的自动化设计

芯片设计流程 eda -O3



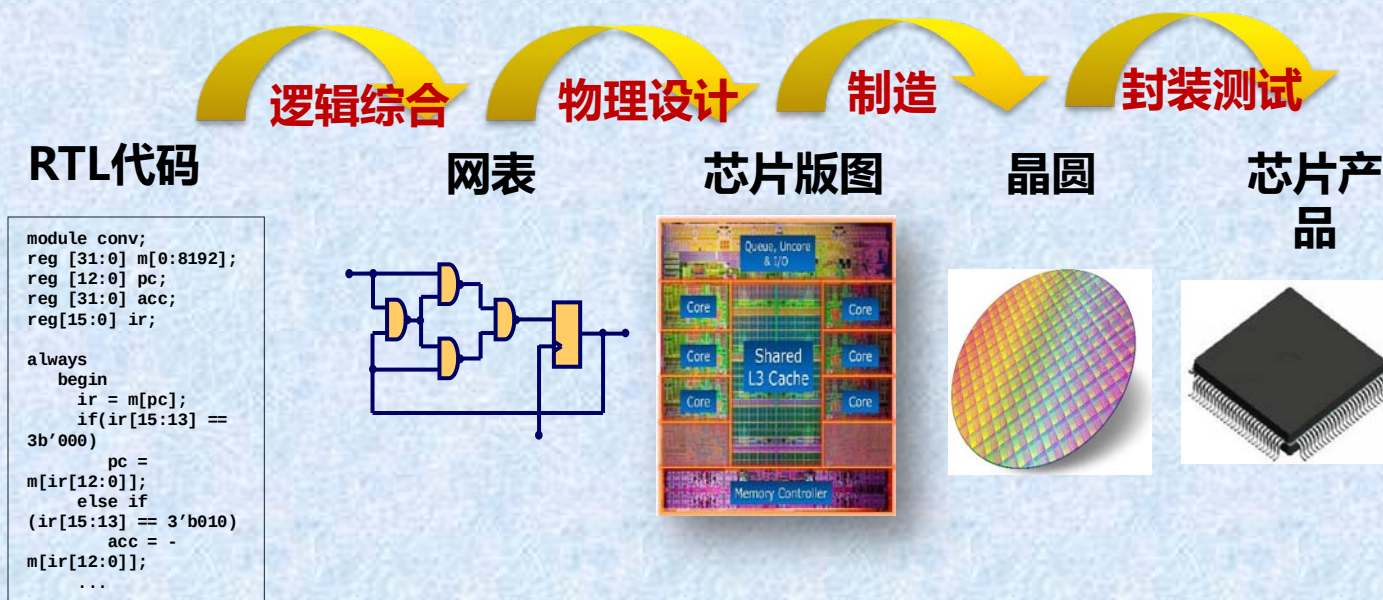
Verilog、Chisel



部分自动化
需要人力，以及专家经验

软硬件开发流程对比：异同点？

- **后端设计**：又称版图设计。用**EDA工具**将设计好功能的**RTL代码**物理化成**GDS版图**
- **芯片制造**：将设计好的版图通过**光刻制造封装**形成**芯片**



EDA工具全程支持后端设计流程

布局布线流程

1、设计设置

Design Setup



2、布局规划

Floorplan



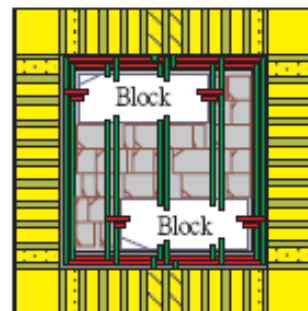
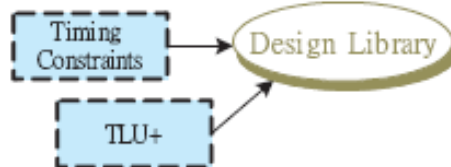
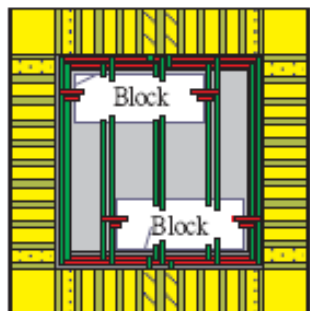
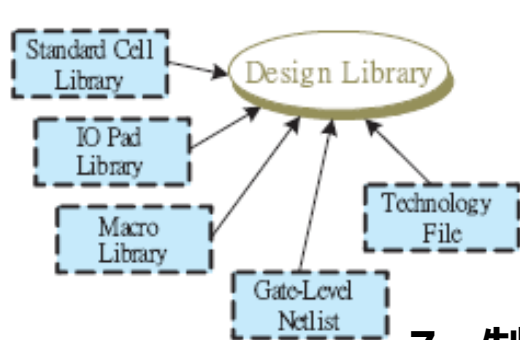
3、时序设置

Timing Setup



4、布局

Placement



Scan Chain Optimization

7、制造设计

Design for Manufacture



6、布线

Routing



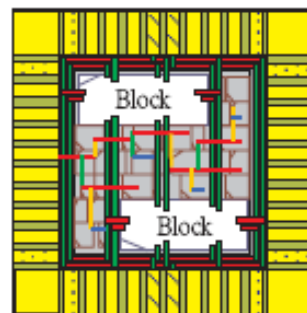
5、时钟树综合

Clock Tree Synthesis

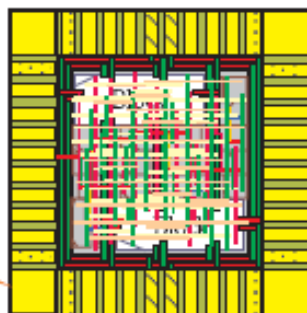


Placement Optimization

Clock Tree Optimization



Routing Optimization



Design Setup（设计设置）：

在这个阶段，设计人员定义芯片的规格、功能和约束。包括电源电压、时钟频率、IO接口等。这些设置将指导后续步骤。

Floorplan（布局规划）：

Floorplan 定义了芯片的尺寸、形状和各个区域的位置。

主要任务包括：

- 确定芯片的尺寸和形状。

- 定义IO单元、填充和角落pad的位置。

- 放置宏单元和标准单元。

- 规划布局 and 布线的障碍物。

- 构建电源地网络。

Timing Setup（时序设置）：

在这一阶段，设计人员设置时序约束，以确保芯片在特定时钟频率下正常工作。这包括设置时钟路径、最小/最大延迟、时序预算等。

Placement（布局）：

将各个基本单元（宏单元和标准单元）在芯片中进行布局。
优化目标是缩短互连线的长度，减小延迟。
考虑电源线、地线、时钟线等的布局。

Clock Tree Synthesis（时钟树综合）：

将时钟信号连接至需要驱动的各个部分。
优化时钟树的结构，以减小时钟延迟。

Routing（布线）：

将各个单元之间的连线完成，包括数据路径和时钟路径。
优化目标是确保信号的可靠传输，同时减小延迟和功耗。

Design for Manufacture（制造设计）：

考虑制造过程中的因素，如工艺变化、电路可靠性等。
优化目标是提高芯片的良品率和可靠性。

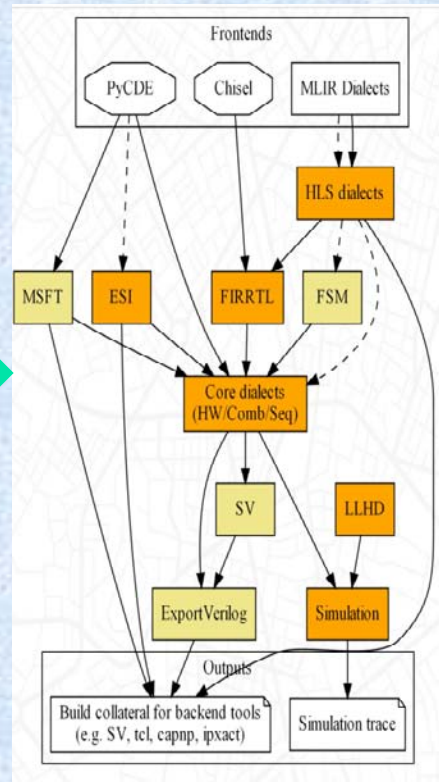
硬件设计描述语言现状

- Verilog、VHDL 是主流的硬件设计语言，现有的EDA后端设计工具都是基于这两者语言。
- 近期高级设计语言和工具的繁荣
- Chisel: 基于Scalar语言描述硬件，并产生基于Verilog的RTL



HDL Modernization

• New HW models / abstraction approaches – metaprogramming / Type systems



CIRCT的动力和特点

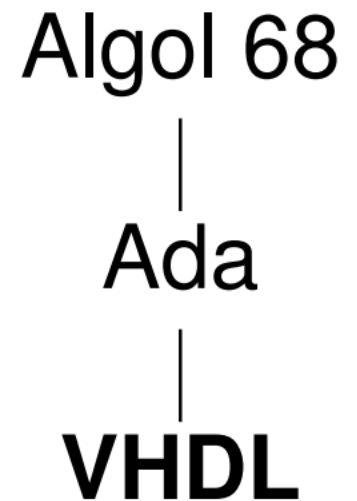
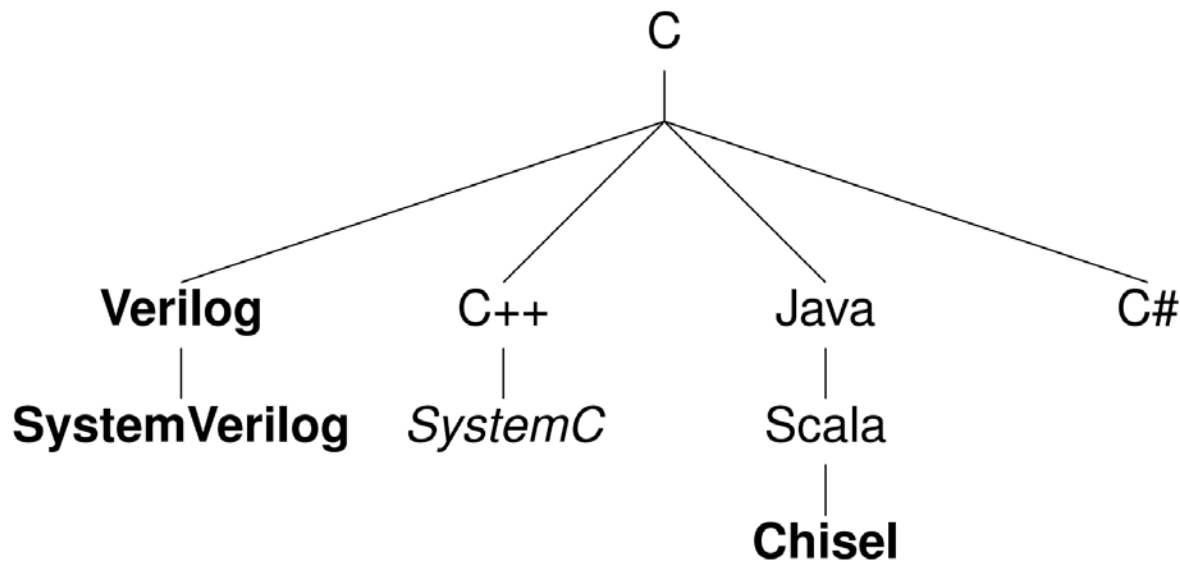
- Circuit IR compiler and tools – 电路编译器工具
- 基于重要的编译框架 LLVM, MLIR
- 统一的框架
 - 模块化：渐进式实现
 - 可扩展：采用标准API实现互操作

设计语言的繁荣 并转向基于MLIR的框架CIRCT
Chisel经过初期阶段开始加速并变得越来越成熟

CIRCT 是一个基于 MLIR (Multi-Level Intermediate Representation) 的开源硬件编译框架。它的目标是为硬件编译器和工具提供与 LLVM 对软件的贡献类似的支持

认识Chisel语言

- 1、**Scala**：**Scala**是一种编程语言，它是**Chisel**的基础。**Chisel**是在**Scala**中定义的特殊类和对象，用于硬件描述。
- 2、**Chisel**：**Chisel**是一个硬件构建语言，允许使用**Scala**语法来生成硬件描述。您可以将**Chisel**代码编译成**FIRRTL**（中间表示）和**Verilog**代码。
- 3、**FIRRTL**是一种中间表示，用于优化和转换硬件描述。



Chisel支持更快的参数化选择和算法设计和迭代

参数化/结构化

- 不同的替换策略
- 队列和缓存配置
- 分支预测结构.....

```
icacheParameters: ICacheParameters = ICacheParameters(
  tagECC = None,
  dataECC = None,
  replacer = Some("setplru"),
  nMissEntries = 2,
  nProbeEntries = 2,
```

```
object ReplacementPolicy {
  def fromString(s: Option[String], n_ways: Int): ReplacementPolicy =
  def fromString(s: String, n_ways: Int): ReplacementPolicy =
    case "random" => new RandomReplacement(n_ways)
    case "lru" => new TrueLRU(n_ways)
    case "plru" => new PseudoLRU(n_ways)
```

实现多种算法，并方便
支持不同配置

参数化配置，表的个数/大小，buffer 容量等

```
PageTableInfos: Seq[Tuple3[Int,Int,Int]] =
  Seq(( 4096, 8, 8),
      ( 4096, 13, 8),
      ( 4096, 32, 8),
      ( 4096, 119, 8)),
```

```
UbtbSize: Int = 256,
FtbSize: Int = 2048,
RasSize: Int = 16,
RasSpecSize: Int = 32,
RasCtrSize: Int = 8,
FtbWays: Int = 8,
```

代码迭代

- 产生复杂控制逻辑
 - 解码器，加法器，浮点
 - 分支预测，选择器，仲裁器
 - 并行，串行处理方式

```
object UIntToOH {
  def apply(in: UInt): UInt = 1.U << in
  def apply(in: UInt, width: Int): UInt = width match {
    case 0 => 0.U(0.W)
    case 1 => 1.U(1.W)
    case _ =>
      val shiftAmountWidth = log2Ceil(width)
      val shiftAmount = in.pad(shiftAmountWidth)(shiftAmountWidth - 1, 0)
      (1.U << shiftAmount)(width - 1, 0)
  }
}
```

```
val u_way = Mux(io.update_write,
val u_mask = UIntToOH(u_way)
```

设计模块
化的函数
并调用

方便快捷实现TAGE, RAS, ITTAGE，支持架构/参数化探索



使用Chisel设计的优点和缺点

设计



验证



不同硬件描述语言特征对比

特性	Chisel	System Verilog	Verilog
结构化大容量连接	Yes	Yes, 有限制	No
元编程	Yes	部分支持不可综合	No
面向对象编程	Yes	部分支持不可综合	No
函数式编程	Yes	No	No

优点：

- 使用高级语言的设计效率得到学术界和工业界设计师的认可
- 可重用性，易于理解，易于连接，参数化设计等

担忧：

- 新语言学习曲线
3-6个月
- 验证问题：RTL可读性问题、不能在Chisel设计中实现验证需要的断言/覆盖
Chisel 6.0的验证在Chisel中得到了类似SVA的支持，大大改善
- DFT和ECO的问题：可以解决
- 与Verilog比较的性能问题
可以通过编码标准和用户控制



开源的RISC-V处理器和Chisel工具链项目

	Github项目名称	设计语言	星级	Fork
开源RISC-V核	Rocketchip	Chisel	2.8K	1K
	Xiangshan	Chisel	4K	549
	Boom	Chisel	1.5k	382
	Pulp	System Verilog	367	104
	VeeR-EH1	Verilog	731	194
Chisel工具链	Chisel	Scala based	3.2k	542
	CIRCT	C++	1.4k	248

实验答疑

- ▶ 课程时间：助教和老师答疑
- ▶ 课程外时间：组建微信群，可在线答疑
- ▶ 答疑方式（统一）：
将问题整理好（配必要的图片）发送到答疑群。